

Reviewer Pack — Kronecker Coefficient Exponential Gap in Symmetric Power Dec...

Entry 65be0d50470d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 08:49:12 UTC

Kronecker Coefficient Exponential Gap in Symmetric Power Decomposition for Permanent vs Determinant

Entry ID: 65be0d50470d Verdict: INCONCLUSIVE Recorded: 2026-05-20 08:49:12 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups **Field B** (complexity object): Resolution Width of Tseitin Formulas on Expanders

Statement:

For all $n \geq 1$, the Kronecker coefficient of the symmetric power of the standard representation of the symmetric group for the permanent is exponentially larger than that for the determinant, implying a super-polynomial lower bound on resolution width for Tseitin formulas derived from the permanent.

Rationale (proposer’s reasoning):

Kronecker coefficients encode the multiplicity of irreducible representations in tensor products, and their exponential growth for permanent vs determinant could reflect structural differences in their algebraic complexity, potentially leading to resolution width lower bounds.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 06ead1a991875daa

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []

        # Generate Tseitin formula for a simple OR gate
        for i in range(n-1):
            clauses.append([variables[i], variables[i+1]])

        # Add the final clause to ensure all variables are true
        clauses.append(variables)

        return variables, clauses

    def generate_expander_graph(n):
        if n < 4:
            raise ValueError("n must be at least 4")

        edges = []
        for i in range(1, n+1):
            j = (i * 2) % n + 1
            edges.append((i, j))

        return edges

    def young_diagram(clauses):
        # Simplified version of generating a Young diagram from clauses
        diagram = []
        for clause in clauses:
            if len(clause) == 1:
                diagram.append([clause[0]])
            else:
                pass
```

```

        diagram.append(sorted(clause, key=lambda x: int(x[1:])))
    return diagram

def kronecker_coefficient(diagram):
    # Simplified version of computing Kronecker coefficient
    n = len(diagram)
    if n == 1:
        return Fraction(1, 1)

    coeff = Fraction(1, 1)
    for i in range(n-1):
        coeff *= Fraction(len(diagram[i]), len(diagram[i+1]))
    return coeff

def symmetric_power_diagram(diagram, k):
    # Simplified version of computing symmetric power diagram
    new_diagram = []
    for _ in range(k):
        new_diagram.extend(diagram)
    return new_diagram

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    try:
        variables, clauses = generate_tseitin_formula(n)
        edges = generate_expander_graph(n)
        diagram = young_diagram(clauses)

        permanent_coeff = kronecker_coefficient(symmetric_power_diagram(diagram, 1))
        determinant_coeff = kronecker_coefficient(symmetric_power_diagram(diagram, 2))

        if permanent_coeff < determinant_coeff:
            raise ValueError("Permanent coefficient must be greater than determinant coefficient")

        results.append(permanent_coeff / determinant_coeff)
    except Exception as e:
        return {
            "metric_name": "kronecker_coefficient_ratio",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": str(e)
        }

metric_value = sum(results) / len(results)
conjecture_holds = all(result > 1 for result in results)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "kronecker_coefficient_ratio",
    "metric_value": metric_value,
    "instances_tested": len(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

```

```

}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

        if trial_result["conjecture_holds"]:
            results.append(trial_result["metric_value"])

    mean = sum(results) / len(results)
    std_dev = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
    support_fraction = len(results) / len(seeds)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

'metric_name': 'kronecker_coefficient_ratio', 'metric_value': Fraction(1, 1), 'instances_tested': 6,
TRIAL: {'metric_name': 'kronecker_coefficient_ratio', 'metric_value': Fraction(1, 1), 'instances_tested': 6,
TRIAL: {'metric_name': 'kronecker_coefficient_ratio', 'metric_value': Fraction(1, 1), 'instances_tested': 6,
TRIAL: {'metric_name': 'kronecker_coefficient_ratio', 'metric_value': Fraction(1, 1), 'instances_tested': 6,
TRIAL: {'metric_name': 'kronecker_coefficient_ratio', 'metric_value': Fraction(1, 1), 'instances_tested': 6,
TRIAL: {'metric_name': 'kronecker_coefficient_ratio', 'metric_value': Fraction(1, 1), 'instances_tested': 6,
TRIAL: {'metric_name': 'kronecker_coefficient_ratio', 'metric_value': Fraction(1, 1), 'instances_tested': 6,
TRIAL: {'metric_name': 'kronecker_coefficient_ratio', 'metric_value': Fraction(1, 1), 'instances_tested': 6,
TRIAL: {'metric_name': 'kronecker_coefficient_ratio', 'metric_value': Fraction(1, 1), 'instances_tested': 6,
TRIAL: {'metric_name': 'kronecker_coefficient_ratio', 'metric_value': Fraction(1, 1), 'instances_tested': 6,
TRIAL: {'metric_name': 'kronecker_coefficient_ratio', 'metric_value': Fraction(1, 1), 'instances_tested': 6,

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c79c869c.py", line 123, in <module>
    mean = sum(results) / len(results)
            ~~~~~^~~~~~
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError before producing valid data, preventing reliable evaluation of conjecture support/falsification. | next: Debug the test code's division by zero error and re-run with proper input handling

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	93320
2	propose	ollama_remote	qwen3:8b	0	0	123023
3	preregistration	ollama_remote	qwen3:8b	0	0	31587
4	novelty	ollama_remote	qwen3:8b	0	0	27633
5	novelty	ollama_remote	qwen3:8b	0	0	22682
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	72039
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12340
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17784
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11912
10	judge	ollama_remote	qwen3:8b	0	0	22085

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 434405 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/65be0d50470d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/65be0d50470d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/65be0d50470d.tar.gz (if generated)